

# 泰坦尼克号生存预测

## 1. 初步判断

给一份泰坦尼克号乘客名单，判断 **这个人最后活下来了没有**

- **输入**：一个乘客的信息（几等舱、性别、年龄、票价、和几个家人同行.....）
- **输出**：`live`（活）或 `death`（死）
- **本质**：这是一个**二分类**问题，模型输出一个概率，大于 0.5 判为活，否则判为死。

**完整流程**：

```
raw_data.csv (891 行原始数据)
|
| 1.data_prepare.py: 清洗 + 切分
train.csv / val.csv / test.csv (625 / 133 / 133 行, 13 列)
prep_params.json (中位数、均值等, 预测时要用)
|
| 2.train.py: 训练 + 画图 + 存模型
outputs/model.pth
docs/loss_curve.png、docs/accuracy_curve.png
|
| ③ predict.py: 读模型, 对新数据出结果
终端打印 Survived (1) / Deceased (0)
```

## 2. 目录结构与运行顺序

```
1  titanic_submission/
2  |— data/
3  |   |— raw_data.csv      891 行原始数据（唯一的数据来源）
4  |   |— train.csv        训练集 625 行（模型从这里学）
5  |   |— val.csv          验证集 133 行（验证模型能力）
6  |   |— test.csv         测试集 133 行（只在最后用一次，看真实水平）
7  |   |— input.csv        要预测的数据（你自己放）
8  |   |— prep_params.json  预处理参数
9  |— src/
10 |   |— data_prepare.py    1. 清洗、算参数、切分
11 |   |— model.py          网络长什么样（只有 18 行）
12 |   |— train.py          2. 训练循环、画图、存模型
13 |   |— predict.py3       加载模型、对 input.csv 出结果
14 |   |— 学习笔记.pdf      字面意思
15 |— docs/
16 |   |— loss_curve.png    损失曲线
17 |   |— accuracy_curve.png 准确率曲线
18 |— outputs/
19 |   |— model.pth         训练好的模型
```

Fence 1

**运行顺序**（必须按这个顺序，因为后一步依赖前一步的文件）：

```
python data_prepare.py
1.生成 train/val/test.csv prep_params.json
python train.py
2.训练, 生成 model.pth + 两张图
python predict.py
3.需要你手动放 input.csv
```

### 3. 数据长什么样

原始数据 `raw_data.csv` : 891 行 × 12 列, 整体存活率 38.38%

| 列名                       | 含义                   | 类型             | 缺失情况             |
|--------------------------|----------------------|----------------|------------------|
| <code>PassengerId</code> | 乘客编号                 | 数字             | 无 (但没用, 编号和生死无关) |
| <code>Survived</code>    | 答案: 0 死 1 活          | 数字             | 无                |
| <code>Pclass</code>      | 舱位等级 1/2/3           | 数字             | 无                |
| <code>Name</code>        | 姓名 (含称呼 Mr/Mrs/Miss) | 文本             | 无                |
| <code>Sex</code>         | 性别                   | 文本 male/female | 无                |
| <code>Age</code>         | 年龄                   | 数字             | 缺 177 个          |
| <code>SibSp</code>       | 兄弟姐妹/配偶 人数           | 数字             | 无                |
| <code>Parch</code>       | 父母/子女 人数             | 数字             | 无                |
| <code>Ticket</code>      | 票号                   | 文本             | 无 (很乱, 弃用)       |
| <code>Fare</code>        | 票价                   | 数字             | 无                |
| <code>Cabin</code>       | 客舱号                  | 文本             | 缺 687 个 (77% 都缺) |
| <code>Embarked</code>    | 登船港口 S/C/Q           | 文本             | 缺 2 个            |

Table 1

#### 3.1 最重要的一条规律: `Sex`

| 性别 | 人数  | 存活率    |
|----|-----|--------|
| 女性 | 314 | 74.20% |
| 男性 | 577 | 18.89% |

Table 2

#### 3.2 瞎猜基线

`train.py` 最后会打印一行 **瞎猜基线 58.65%**。含义是: 如果一个模型什么都不学, 只会背"所有人的答案都是死", 那它在测试集上能对 58.65% (因为测试集里 41.35% 活、58.65% 死, 全猜多的那一边)

**这就是及格线**

测试集准确率 80.45% 的意义 = 比瞎猜高 21.8 个百分点

### 4. PyTorch 基础概念

#### 4.1 Tensor (张量)

**是什么:** PyTorch 里的多维数字数组。类别上它和 numpy 的 `ndarray` 是同类东西, 但 Tensor 多两个能力: 1.能放到 GPU 上算; 2.自动求导

```
1 a = torch.tensor([[1.0, 2.0], [3.0, 4.0]]) # 形状 (2, 2)
2 a.shape      # torch.Size([2, 2])
3 a.dtype      # torch.float32
```

Fence 2

本项目里数据的形状约定:

| 变量   | 形状        | 含义                |
|------|-----------|-------------------|
| x    | (625, 12) | 625 个乘客，每个 12 个特征 |
| y    | (625,)    | 625 个答案，一维        |
| 模型输出 | (625,)    | 625 个概率           |

Table 3

为什么y必须是一维：

如果y是 (625, 1)，算损失时会报

```
Target size (torch.Size([625])) must be the same as input size (torch.Size([625, 1]))
```

AI大人tip:

把 model.py 里 forward 最后加了个 .squeeze(-1)

把 (N, 1)压成 (N,) 解决报错

## 4.2 广播与向量化

是什么：让形状不同的 Tensor 直接做运算，PyTorch 自动把小的"撑开"。

```
1 x = torch.tensor([[1.0, 2.0],
2                   [3.0, 4.0]])      # (2, 2)
3 m = torch.tensor([10.0, 20.0])     # (2,) → 自动看成 (1, 2) 再撑成 (2, 2)
4 x + m
5 # tensor([[11, 22],
6          [13, 24]])
```

Fence 3

本项目里 out["SibSp"] + out["Parch"] + 1 也是同一个道理

## 4.3 nn.Module (模型的基类)

是什么：PyTorch 里所有神经网络都要继承的父类。继承它以后，你的类自动获得三个能力：

- 1. 能记录自己包含哪些"层" (那些 nn.Linear)
- 2. .parameters() 能列出所有待训练的数字
- 3. .state\_dict() / .load\_state\_dict() 能存 / 取这些数字

继承的写法：class TitanicNet(nn.Module):，并且 \_\_init\_\_ 第一行必须 super().\_\_init\_\_() (这行是在继承父类，不写 PyTorch 会报错)

| 方法       | 什么时候调用          | 干什么           |
|----------|-----------------|---------------|
| __init__ | 创建对象时 一次        | 定义有哪些层、长什么样   |
| forward  | 每次 model(x) 无数次 | 定义数据怎么从输入流到输出 |

Table 4

## 4.4 训练三件套

| 名字   | 是什么                                   | 本项目用的                  |
|------|---------------------------------------|------------------------|
| 损失函数 | 一个数字，衡量"这次猜得有多离谱"。越小越好                | nn.BCEWithLogitsLoss() |
| 优化器  | 根据损失，决定 225 个参数每个该往哪边调、调多少            | torch.optim.Adam       |
| 梯度   | 每个参数"往哪调能让损失变小"的方向，由 .backward() 自动算出 | 自动求导                   |

Table 5

为什么用 `BCEWithLogitsLoss` 而不是 `BCELoss` : 二分类交叉熵。`WithLogits` 版本的意思是"我把 sigmoid 一起打包了"—你直接喂原始输出 (叫 `logit`) , 它内部先 `sigmoid` 再算交叉熵

4.5 `torch.no_grad()` 和 `model.eval()`

| 名字                           | 是什么               | 什么时候用                               |
|------------------------------|-------------------|-------------------------------------|
| <code>torch.no_grad()</code> | 一个"临时关掉自动求导记录"的开关 | 评估、预测时。省内存、更快                       |
| <code>model.eval()</code>    | 把模型切到"评估模式"       | 评估、预测时。会影响 <code>Dropout</code> 的行为 |

Table 6

**Dropout** : 训练时它会随机把一部分神经元临时"关掉" (本项目 `DROPOUT=0.2` ) , 逼模型别太依赖某几个神经元, 防止死记硬背。预测时必须关掉这个随机性——这就是 `model.eval()` 干的事。如果预测时忘了写 `model.eval()` , 结果会每次都不同

5. `data_prepare.py` 思路 (数据处理思路)

这个文件分四块: 路径 → 参数 → `clean()` 函数 → `main()` 。

5.1 路径

```
1 SRC = os.path.dirname(os.path.abspath(__file__))
2 PROJECT = os.path.dirname(SRC)
3 DATA = os.path.join(PROJECT, "data")
```

Fence 4

`__file__` 是 Python 自动提供的一个变量, 值是"当前这个 `.py` 文件的路径"  
`os.path.abspath()` 把相对路径变成绝对路径 ( `C:\...` )  
`os.path.dirname()` 去掉最后一段, 得到"所在文件夹"

所以 `SRC` = `src` 文件夹, `PROJECT` = 项目根, `DATA` = 数据文件夹。

为什么这么写而不是直接写 `"data/raw_data.csv"` : 直接写相对路径的话, 你在不同目录下运行时 ( `cd src` 还是 `cd 项目根` ) 会找错地方。用 `__file__` 算出来的是绝对路径, 在哪运行都对

5.2 参数区

```
1 SEED = 42
2 VAL_RATIO = 0.15
3 TEST_RATIO = 0.15
4 N_FEATURES = 12
5 EPOCHS = 300
6 BATCH_SIZE = 32
7 LR = 1e-3
8 WEIGHT_DECAY = 1e-4
9 DROPOUT = 0.2
10 HIDDEN = 16
```

Fence 5

| 名字                     | 含义               | 为什么是这个值                         |
|------------------------|------------------|---------------------------------|
| SEED                   | 随机种子             | 固定随机性，让结果可复现                    |
| VAL_RATIO / TEST_RATIO | 验证/测试集占 15%      | $891 \times 0.15 \approx 133$ 行 |
| N_FEATURES             | 输入 12 个特征        | 和 FEATURE_COLS 的长度必须一致          |
| EPOCHS                 | 把训练集完整过 300 遍    | 太少学不会，太多会背答案                    |
| BATCH_SIZE             | 每 32 个样本更新一次参数   | $625 / 32 \approx 20$ 次更新算一轮    |
| LR                     | 学习率，每次调参数的"步子大小" | $1e-3 = 0.001$ ，Adam 的常用值       |
| WEIGHT_DECAY           | 惩罚参数变得过大         | 防过拟合                            |
| DROPOUT                | 随机丢弃 20% 神经元     | 防过拟合                            |
| HIDDEN                 | 隐藏层宽度 16         | 数据集小，16 够用，大了就背答案               |

Table 7

### 5.3 TITLE\_MAP：从姓名里挖特征

```
1 TITLE_MAP = {"Mr": 0, "Miss": 1, "Mrs": 2, "Master": 3, ...}
```

Fence 6

原始数据里有 17 种称呼，但很多只出现 1~2 次（Jonkheer、Countess 各只有 1 个人）。样本只有 1 个的类别，模型根本学不到规律，反而添乱，所以按存活率相近的原则归成 6 类：

| 编号 | 包含的称呼                                        | 人数  | 存活率   |
|----|----------------------------------------------|-----|-------|
| 0  | Mr（成年男性）                                     | 517 | 15.7% |
| 1  | Miss / Ms / Mlle                             | 185 | 70.3% |
| 2  | Mrs / Mme                                    | 126 | 79.4% |
| 3  | Master（小男孩）（糟了有神父）                           | 40  | 57.5% |
| 4  | Dr / Rev（医生、牧师）                              | 13  | 23.1% |
| 5  | 贵族/军官（Col、Major、Capt、Sir、Lady、Countess.....） | 10  | 50.0  |

Table 8

### 5.4 clean() 十一个特征逐个拆

函数的输入输出：

|    | 内容                                    |
|----|---------------------------------------|
| 输入 | df（原始表，891×12）、pm（参数字典，5 个键）          |
| 输出 | 新的 DataFrame，13 列 = 12 个特征 + Survived |

Table 9

ai大人tip：

函数里没有一行修改 df，而是新建了一个 out 表往里填

这样原始数据永远完好，出错了可以直接重跑

| #   | 代码                                                                   | 干了什么          | 为什么                                                                                        |
|-----|----------------------------------------------------------------------|---------------|--------------------------------------------------------------------------------------------|
| 1   | <code>out["Pclass"] = df["Pclass"]</code>                            | 直接搬           | 1/2/3 本身就有大小含义，不用改                                                                         |
| 2   | <code>.map({"male": 0, "female": 1})</code>                          | 文本转数字         | 模型只能吃数字                                                                                    |
| 3   | <code>"Age".fillna(中位数)</code>                                       | 177 个缺失填 28.0 | 填均值会被极端值带偏，中位数更稳                                                                           |
| 4-5 | <code>SibSp</code> 、 <code>Parch</code> 直接搬                          | 滚木            | 滚木                                                                                         |
| 6   | <code>"Fare".fillna(中位数)</code>                                      | 无缺失，但保险       | 滚木                                                                                         |
| 7   | <code>fillna("S")</code> 再<br><code>.map({"S":0,"C":1,"Q":2})</code> | 2 个缺失填众数 S    | 众数 = 出现最多的，S 最多                                                                            |
| 8   | <code>df["Cabin"].notnull().astype(int)</code>                       | 有客舱号=1，没有=0   | 原始 <code>Cabin</code> 缺 77%，直接用会丢掉大部分数据但 <b>“有没有客舱号”本身就是强信号</b> ——有客舱号说明能够被记录（要知道死人是不会说话的） |
| 9   | <code>SibSp + Parch + 1</code>                                       | 全家总人数         | 两个特征合成一个更有意义的： <code>FamilySize</code>                                                     |
| 10  | <code>(FamilySize == 1).astype(int)</code>                           | 是不是独自一人       | 独行者存活率明显不同                                                                                 |
| 11  | <code>str.extract(r" ([A-Za-z]+)\.")</code>                          | 正则从姓名里抓称呼     | 看上面                                                                                        |
| 12  | <code>(Age &lt;= 12).astype(int)</code>                              | 是不是小孩         | 小孩优先上救生艇                                                                                   |
| 13  | <code>(Fare - mean) / std</code>                                     | 标准化           | 看下面                                                                                        |

Table 10

为什么 `Fare` 要标准化：

原始票价从 0 到 512，跨度极大；而 `Pclass` 只有 1/2/3，`IsChild` 只有 0/1

如果不处理，**票价这个数字会在计算中占绝对主导**，模型会以为票价最重要

标准化之后，所有特征的量级差不多（均值 0、标准差 1），模型才能公平地看待每一个。

**注意 `Fare` 被处理了两次**：第 6 行先填缺失，第 13 行才标准化。顺序不能反——先标准化的话，缺失值 `NaN` 参与计算会把均值和标准差也变成 `NaN`。

`astype(int)` 必须放在 `fillna` 之后：`.map()` 遇到字典里没有的值会返回 `NaN`，直接 `astype(int)` 会报 `IntCastingNaNError`

`Title` 那行特意加了 `.fillna(0)` 兜底（万一出现没见过的称呼，当 Mr 处理）

`(条件).astype(int)` 这个写法：`Age <= 12` 得到一列 `True/False`，`.astype(int)` 把 `True` 变 1、`False` 变 0。这是 pandas 里造 0/1 特征的固定套路。

## 5.5 老大我们怎么算啊

```

1 pm = {
2     "age_median": float(df["Age"].median()),      # 28.0
3     "fare_median": float(df["Fare"].median()),    # 14.4542
4     "fare_mean": float(df["Fare"].mean()),        # 32.204
5     "fare_std": float(df["Fare"].std()),           # 49.693
6     "embarked_mode": str(df["Embarked"].mode()[0]) # "S"
7 }
```

以防你不知道：

`mode()` 返回的是一个 Series (因为可能并列多个众数)，所以 `[0]` 取第一个。外面套 `float()` / `str()` 是为了转成 Python 原生类型 — numpy 的数字类型不能直接存进 JSON (还有第二关)

## 5.6 ? ! 水果忍者? ! (我切切切)

```
1 n = len(out) # 891
2 idx = np.random.permutation(n) # 0~890 随机打乱后的顺序
3 n_test = int(n * TEST_RATIO) # 133
4 n_val = int(n * VAL_RATIO) # 133
5
6 test_idx = idx[:n_test] # 前 133 个 → 测试集
7 val_idx = idx[n_test:n_test + n_val] # 接着 133 个 → 验证集
8 train_idx = idx[n_test + n_val:] # 剩下 625 个 → 训练集
```

Fence 8

三个集的分工：

| 集合    | 行数  | 模型见过吗          | 干什么用           |
|-------|-----|----------------|----------------|
| train | 625 | 见过答案，就是用来学的    | 调参数            |
| val   | 133 | 没见过            | 训练途中用来挑"最好的一版" |
| test  | 133 | 没见过，且全程不参与任何决策 | 最后汇报成绩         |

Table 11

`reset_index(drop=True)` 的作用：

切分后行号是乱的 (比如 3, 7, 12...)。重置成 0,1,2...

## 6. model.py

```
1 import torch.nn as nn
2 from data_prepare import HIDDEN, DROPOUT
3
4
5 class TitanicNet(nn.Module):
6     def __init__(self, n_features):
7         super().__init__()
8         self.net = nn.Sequential(
9             nn.Linear(n_features, HIDDEN), # 12 → 16
10            nn.ReLU(),
11            nn.Dropout(DROPOUT),
12            nn.Linear(HIDDEN, 1), # 16 → 1
13        )
14        self.criterion = nn.BCEWithLogitsLoss()
15
16    def forward(self, x):
17        return self.net(x).squeeze(-1)
```

Fence 9

### 6.1 nn.Sequential 是什么

一个"流水线容器"。你按顺序放几层进去，数据就会依次穿过它们。等价于：

```
1 def forward(self, x):
2     x = self.net[0](x)
3     x = self.net[1](x)
4     x = self.net[2](x)
5     return self.net[3](x)
```

Fence 10

用 `Sequential` 写更短，而且层多了也不容易看错顺序

## 6.2 四层各自的作用

| 层                              | 形状变化                          | 作用                                                                                    |
|--------------------------------|-------------------------------|---------------------------------------------------------------------------------------|
| <code>nn.Linear(12, 16)</code> | $(N, 12) \rightarrow (N, 16)$ | <b>全连接层</b> 。16 个神经元各自给 12 个特征配一组权重 ( $12 \times 16 = 192$ 个权重 + 16 个偏置)。这是真正"学规律"的地方 |
| <code>nn.ReLU()</code>         | $(N, 16) \rightarrow (N, 16)$ | <b>激活函数</b> ：它提供了"非线性"，把负数变 0，正数不变。 <b>没有它，两个 Linear 叠起来数学上等于一个 Linear</b>            |
| <code>nn.Dropout(0.2)</code>   | 不变                            | 训练时随机丢掉 20% 的神经元，防止死记硬背                                                               |
| <code>nn.Linear(16, 1)</code>  | $(N, 16) \rightarrow (N, 1)$  | 把 16 个数汇总成 1 个数（这个数就是 logit，还没 sigmoid）                                               |

Table 12

## 6.3 `squeeze(-1)` 为什么必须有

`Linear(16, 1)` 的输出形状是  $(N, 1)$ ，但 `y` 的形状是  $(N,)$ 。损失函数要求两者形状一致，否则报：

```
1 ValueError: Target size (torch.Size([4])) must be the same as input size (torch.Size([4, 1]))
```

Fence 11

`.squeeze(-1)` 的意思是"把**最后一维**（`-1` 表示倒数第一维）里长度为 1 的维度去掉"， $(N, 1) \rightarrow (N,)$

# 7. `train.py` 构建思路

## 7.1 `set_seed()`：固定三个随机源

```
1 def set_seed(seed):
2     random.seed(seed)          # Python 自带的 random
3     np.random.seed(seed)       # numpy 的随机
4     torch.manual_seed(seed)    # PyTorch 的随机
```

Fence 12

**为什么三个都要设**：它们是**三套独立的随机数生成器**，设一个不影响另外两个。本项目里三个都用到了（切分用 `numpy`、`DataLoader` 用 `torch`、`Python` 的 `random`没用上）

## 7.2 `read_csv()`：从 CSV 到 Tensor

```
1 def read_csv(name):
2     df = pd.read_csv(os.path.join(DATA, f"{name}.csv"))
3     X = df.iloc[:, :N_FEATURES].to_numpy(dtype=np.float32)
4     y = df[LABEL_COL].to_numpy(dtype=np.float32)
5     return torch.from_numpy(X).float(), torch.from_numpy(y).float()
```

Fence 13

| 部分 | 是什么                                                                                     |
|----|-----------------------------------------------------------------------------------------|
| 输入 | <code>name</code> ：字符串， <code>"train"</code> / <code>"val"</code> / <code>"test"</code> |
| 输出 | <b>两个 Tensor</b> ： <code>x</code> 形状 $(N, 12)$ ， <code>y</code> 形状 $(N,)$               |

Table 13

- `df.iloc[:, :12]` — `iloc` 是"按位置取"。`:` 表示所有行，`:12` 表示**前 12 列**。第 13 列是 `Survived`，正好被排除在外。**这就是为什么 `data_prepare.py` 里特征列的顺序不能乱**
- `dtype=np.float32` — 显式转成 32 位浮点。`PyTorch` 的 `Linear` 层默认只接受 `float32`，用 `float64` 会报类型不匹配（怎么还有这个bug）

- `torch.from_numpy(...)` — 把 numpy 数组"包"成 Tensor。它和原数组 **共享同一块内存**（不复制），所以很快（依旧ai大人指导老师）
- 最后那个 `.float()` 是保险（`from_numpy` 已经保证是 float32 了）

## 7.3 `TensorDataset` + `DataLoader`：批处理的机制

```
1 loader = DataLoader(
2     TensorDataset(Xtr, ytr),
3     batch_size=BATCH_SIZE,
4     shuffle=True,
5     generator=torch.Generator().manual_seed(SEED)
6 )
```

Fence 14

| 名字                               | 是什么                     | 输入 → 输出                                                                |
|----------------------------------|-------------------------|------------------------------------------------------------------------|
| <code>TensorDataset(X, y)</code> | 把 X 和 y <b>按行配对</b> 的容器 | <code>X(625, 12)</code> + <code>y(625, )</code> → 625 个"（一行特征，一个答案）"组合 |
| <code>DataLoader</code>          | <b>分批器</b>              | 625 个组合 → 每轮吐出 20 个批次，每批 32 个                                          |
| <code>shuffle=True</code>        | 每轮开始前重新打乱顺序             | 防止模型记住"第 1 个样本总是死"这种顺序规律                                               |
| <code>generator=...</code>       | 给打乱指定种子                 | 让打乱也可复现                                                                |

Table 14

训练循环里怎么用：

```
1 for xb, yb in loader:      # 自动解包成一批一批的
2     # xb: (32, 12)    yb: (32,)
```

Fence 15

**为什么要分批而不是一次性喂 625 个**：1.内存友好，数据量大时尤其重要；2.每批更新一次参数（ $625/32 \approx 20$  次/轮，300 轮共 6000 次），比"一轮只更新 1 次"收敛快得多

以防你不知道 `TensorDataset` 可以自动处理余数（

## 7.4 `evaluate()`：算损失和准确率

```
1 @torch.no_grad()
2 def evaluate(model, X, y):
3     model.eval()
4     logits = model(X)
5     loss = model.criterion(logits, y).item()
6     preds = (torch.sigmoid(logits) > 0.5).float()
7     acc = (preds == y).float().mean().item()
8     return loss, acc
```

Fence 16

逐行：

| 行                                           | 做什么                                                     |
|---------------------------------------------|---------------------------------------------------------|
| <code>@torch.no_grad()</code>               | <b>装饰器</b> ：自动给整个函数加上“不求导”的开关，函数内所有运算都不记录梯度（省内存、更快）     |
| <code>model.eval()</code>                   | 关掉 Dropout 的随机性                                         |
| <code>logits = model(X)</code>              | 前向，拿到原始输出（没 sigmoid）                                    |
| <code>model.criterion(logits, y)</code>     | 算损失， <code>.item()</code> 把单个数字的 Tensor 转成 Python float |
| <code>torch.sigmoid(logits) &gt; 0.5</code> | 转概率、和 0.5 比，得到一系列 <code>True/False</code>               |
| <code>.float()</code>                       | <code>True/False</code> → <code>1.0/0.0</code>          |
| <code>(preds == y).float().mean()</code>    | 逐元素比较，转成 1/0，求平均 = <b>准确率</b>                           |

Table 15

`(preds == y)` 得到 `[True, False, True, ...]`，`.mean()` 就得到 0.667 这种比例——这是 PyTorch 里算准确率的标准写法（`True` 当 1，`False` 当 0）

**注意** `evaluate` 不会把模型切回训练模式。所以训练循环里每轮开头必须手动 `model.train()`，否则第 2 轮开始 Dropout 就一直被关着了

## 7.5 训练循环：公式化的五行

```

1  for epoch in range(1, EPOCHS + 1):
2      model.train()
3      total_loss = 0.0
4      for xb, yb in loader:
5          optimizer.zero_grad()                # 1
6          loss = model.criterion(model(xb), yb)  # 2
7          loss.backward()                       # 3
8          optimizer.step()                     # 4
9          total_loss += loss.item() * len(xb)   # 5
10     scheduler.step()
11     train_loss = total_loss / len(Xtr)

```

Fence 17

这五行是所有 PyTorch 训练的固定套路，顺序绝对不能错：

| # | 代码                                          | 干什么                    | 不写会怎样                                          |
|---|---------------------------------------------|------------------------|------------------------------------------------|
| 1 | <code>optimizer.zero_grad()</code>          | 清空上一批的梯度               | PyTorch 的梯度是累加的。不清空的话，这一批的梯度会加上上一批的，越滚越大，训练直接崩 |
| 2 | <code>model.criterion(model(xb), yb)</code> | 前向 + 算损失               |                                                |
| 3 | <code>loss.backward()</code>                | 反向传播，自动算出 225 个参数各自的梯度 | 不写 <code>step()</code> 没东西可用                   |
| 4 | <code>optimizer.step()</code>               | 按梯度真正修改 225 个数         | 参数不变，白白训练                                      |
| 5 | <code>+= loss.item() * len(xb)</code>       | 累积损失                   |                                                |

Table 16

**2和4的分工**：`backward()` 只计算“该往哪调”（把结果存在每个参数的 `.grad` 里），`step()` 才真的调。中间隔着这两步，是为了让优化器有机会做额外处理（比如 Adam 的动量和 `weight_decay`）

**为什么乘 `len(xb)`**：最后一批不足 32 个（ $625 = 19 \times 32 + 17$ ），如果直接累加每批的平均值再除以批次数，最后那小批的权重就被高估了

7.6 scheduler：学习率衰减

```
1 scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
2 ...
3 scheduler.step()      # 每轮调用一次
```

Fence 18

**是什么：**让学习率从 1e-3 渐渐降到接近 0，走一条余弦曲线

**为什么需要：**训练初期参数离目标远，步子大一点（1e-3）进展快；训练后期已经很接近最优解了，步子还大的话会在最优点附近反复横跳，永远落不下去。衰减让后期步子越来越小，稳稳收住。

`T_max=EPOCHS` 表示"用 300 轮走完半个余弦周期"

7.7 保存最好的一版

```
1 best_acc, best_state = -1.0, None
2 for epoch in ...:
3     ...
4     if val_acc > best_acc:
5         best_acc = val_acc
6         best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
7
8 model.load_state_dict(best_state)
```

Fence 19

**为什么需要这个：**训练 300 轮，最后一轮不一定是最好的一轮。验证集准确率通常是先升后波动，最后可能反而降了（过拟合）

所以要在训练途中记下"验证集表现最好的那一刻"的参数

`{k: v.detach().clone() for k, v in model.state_dict().items()}` 逐个解释：

| 部分                              | 作用                                                                     |
|---------------------------------|------------------------------------------------------------------------|
| <code>model.state_dict()</code> | 一个字典， <code>{"net.0.weight": Tensor, "net.0.bias": Tensor, ...}</code> |
| <code>.items()</code>           | 遍历"键，值"                                                                |
| <code>.detach()</code>          | 从计算图里摘出来，不再记录梯度（不摘会连带一大坨计算历史，占内存）                                      |
| <code>.clone()</code>           | 复制一份。不复制的话存的是引用，下一轮参数一变，"快照"跟着变，等于没存                                   |

Table 17

7.8 plot()：两张曲线图

```
1 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
2 plt.rcParams["axes.unicode_minus"] = False
```

Fence 20

- 第一行：给 matplotlib 指定中文字体，否则图里的中文会变成方框（用不上吧，就当ai大人太贴心了）
  - 第二行：`unicode_minus = False`，让负号用 ASCII 的 - 而不是 Unicode 的 −，否则坐标轴上的负号也会变方框
- `matplotlib.use("Agg")`（在文件开头，必须在 `import pyplot` 之前）：指定"不弹窗的后端"，图片直接存成文件

两张图的区别就是画的曲线不同

| 图                               | 曲线                  | 怎么看                                 |
|---------------------------------|---------------------|-------------------------------------|
| <code>loss_curve.png</code>     | train_loss、val_loss | 两条都下降 = 在学；train 一直降但 val 开始升 = 过拟合 |
| <code>accuracy_curve.png</code> | train_acc、val_acc   | 两条曲线必须画在同一张图上才能对比                   |

Table 18

## 7.9 save\_model(): 存什么

```
1 torch.save({"state_dict": model.state_dict(), "n_features": N_FEATURES}, MODE
```

Fence 21

存一个字典，两样东西：

| 键          | 为什么必须存                                                                      |
|------------|-----------------------------------------------------------------------------|
| state_dict | 225 个参数的数值                                                                  |
| n_features | 重建网络结构需要知道输入是 12 维。只存参数的话，predict.py 不知道该用 TitanicNet(12) 还是 TitanicNet(20) |

Table 19

加载时 weights\_only=False 是 PyTorch 2.6+ 的要求（默认 True 只允许加载纯张量，字典里带字符串会报错）

## 8. predict.py

### 8.1 对要预测的csv复用训练时的 clean() 和 prep\_params.json

```
1 from data_prepare import clean, PARAM_JSON, MODEL, FEATURE_COLS, DATA
2 ...
3 with open(PARAM_JSON, "r", encoding="utf-8") as f:
4     pm = json.load(f)
5     out = clean(df, pm)
```

Fence 22

它做了两件事：

1. 调同一个 clean() 函数 — 保证新数据的 12 个特征是用完全一样的方式算出来的。
2. 读训练时存下的 pm — 保证填补缺失用的是训练集的中位数 28.0，而不是新数据自己的中位数

### 8.2 丢掉答案列

```
1 df = df.drop(columns=["Survived"], errors="ignore")
```

Fence 23

- errors="ignore"：如果这一列不存在，不报错直接跳过。所以这个 predict.py 对"带答案的数据"和"不带答案的数据"都能用
- 为什么必须丢：clean() 会把 Survived 一起带进结果。如果不丢，out 里就有答案了

### 8.3 加载模型

```
1 ckpt = torch.load(MODEL, weights_only=False)
2 model = TitanicNet(ckpt["n_features"]) # 先建一个"空壳"
3 model.load_state_dict(ckpt["state_dict"]) # 再把参数灌进去
4 model.eval() # 切评估模式
```

Fence 24

顺序不能反：load\_state\_dict 要求"空壳的结构"和"参数的形状"一一对应。所以必须先用 n\_features 建出正确形状的网络，才能灌参数。

model.eval() 必须有：否则 Dropout 还在随机丢神经元，同一个乘客每次跑出来的结果可能不一样

8.4 前向 + 判断

```
1 x = torch.from_numpy(out[FEATURE_COLS].to_numpy(dtype=np.float32))
2 with torch.no_grad():
3     probability = torch.sigmoid(model(x)).numpy()
4
5 for i in range(len(out)):
6     if probability[i] > 0.5:
7         print("live")
8     else:
9         print("death")
```

Fence 25

| 行                  | 说明                                         |
|--------------------|--------------------------------------------|
| out[FEATURE_COLS]  | 只取 12 个特征列，顺序和训练时完全一致（FEATURE_COLS 是同一个列表） |
| model(x)           | 前向，输出 logit（没 sigmoid）                     |
| torch.sigmoid(...) | 转成 0~1 的概率                                 |
| > 0.5              | 二分类的默认阈值                                   |

Table 20

9. 实验结果

9.1 seed=42 的完整结果

|   |        |        |
|---|--------|--------|
| 1 | 参数量    | 225    |
| 2 | 训练集准确率 | 83.36% |
| 3 | 验证集准确率 | 84.21% |
| 4 | 测试集准确率 | 80.45% |
| 5 | 瞎猜基线   | 58.65% |

Fence 26

| 指标     | 数值     | 怎么解读                   |
|--------|--------|------------------------|
| 参数量    | 225    | 远小于 625 个训练样本，不易过拟合    |
| 测试集准确率 | 80.45% | 比瞎猜基线高 21.8 个百分点，应该看这个 |

Table 21